



SoC Integration Using the GEM_GXL Ethernet Controller Demo Testbench

Application Note

Revision 0.1

CADENCE CONFIDENTIAL

Confidentiality Notice

© 1996-2017 Cadence Design Systems, Inc. All rights reserved.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 800.862.4522. All other trademarks are the property of their respective holders.

Restricted Permission: This document is protected by copyright law and international treaties and contains trade secrets and proprietary information owned by Cadence. Unauthorized reproduction or distribution of this document, or any portion of it, may result in civil and criminal penalties. Except as specified in this permission statement, this document may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. This document contains the proprietary and confidential information of Cadence or its licensors, and is supplied subject to, and may be used only in accordance with, a written agreement between Cadence and its customer.

Unless otherwise agreed to by Cadence in writing, this statement grants Cadence customers permission to print one (1) hard copy of this document subject to the following conditions:

1. This document may not be modified in any way.
2. Any authorized copy of this document or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement.
3. The information contained in this document cannot be used in the development of like products or software, whether for internal or external use, and shall not be used for the benefit of any other party, whether or not for consideration.

Disclaimer: INFORMATION IN THIS DOCUMENT IS SUBJECT TO CHANGE WITHOUT NOTICE AND DOES NOT REPRESENT A COMMITMENT ON THE PART OF CADENCE. EXCEPT AS MAY BE EXPLICITLY SET FORTH IN A WRITTEN AGREEMENT BETWEEN CADENCE AND ITS CUSTOMER, CADENCE DOES NOT MAKE, AND EXPRESSLY DISCLAIMS, ANY REPRESENTATIONS OR WARRANTIES AS TO THE COMPLETENESS, ACCURACY OR USEFULNESS OF THE INFORMATION CONTAINED IN THIS DOCUMENT. CADENCE DOES NOT WARRANT THAT USE OF SUCH INFORMATION WILL NOT INFRINGE ANY THIRD PARTY RIGHTS, AND CADENCE DISCLAIMS ALL IMPLIED WARRANTIES, INCLUDING MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. CADENCE DOES NOT ASSUME ANY LIABILITY FOR DAMAGES OR COSTS OF ANY KIND THAT MAY RESULT FROM USE OF SUCH INFORMATION. CADENCE CUSTOMER HAS COMPLETE CONTROL AND FINAL DECISION-MAKING AUTHORITY OVER ALL ASPECTS OF THE DEVELOPMENT, MANUFACTURE, SALE AND USE OF CUSTOMER'S PRODUCT, INCLUDING, BUT NOT LIMITED TO, ALL DECISIONS WITH REGARD TO DESIGN, PRODUCTION, TESTING, ASSEMBLY, QUALIFICATION, CERTIFICATION, INTEGRATION OF CADENCE PRODUCTS, INSTRUCTIONS FOR USE, LABELING AND DISTRIBUTION, AND CADENCE EXPRESSLY DISAVOWS ANY RESPONSIBILITY WITH REGARD TO ANY SUCH DECISIONS REGARDING CUSTOMER'S PRODUCT.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227- 14 and DFAR252.227-7013 et seq. or its successor.

Document Revision History

Revision	Date	Changes made in document from previous revision
Rev 0.1	08/11/2017	Initial version.

Table of Contents

List of Acronyms	5
1. Introduction	6
1.1. References	6
2. Architecture	7
2.1. C Layer Reuse in UVM-SV	9
2.2. C Layer Reuse in SoC	11

List of Acronyms

Acronym	Meaning
AMBA	Advanced Microcontroller Bus Architecture
APB	AMBA Peripheral Bus
API	Application Programming Interface
AXI	AMBA eXtensible Interface
CSP	Configurable System Platform
DPI	Direct Programming Interface
GEM	Gigabit Ethernet MAC
IP	Intellectual Property
SoC	System on Chip
SV	SystemVerilog
UVM	Universal Verification Methodology
VE	Verification Environment

1. Introduction

Besides conventional UVM-based tests, the GEM_GXL Demo Testbench is provided with C tests in order to demonstrate basic health checking of the IP using a software-oriented approach. This application note contains details on how a customer can reuse the C layer into a CPU-based SoC VE, taking the Demo Testbench as an example.

With reference to Figure 1, three flows are supported to run in the GEM_GXL Demo Testbench:

1. UVM-SV simulations: this is the conventional way of running UVM tests and will be not further addressed here.
2. C-based simulations w/o the core driver: this flow demonstrates how to use simple C stimulus.
3. C-based simulations w/ the core driver: this flow demonstrates how core driver tests (bare-metal architecture) can be integrated.

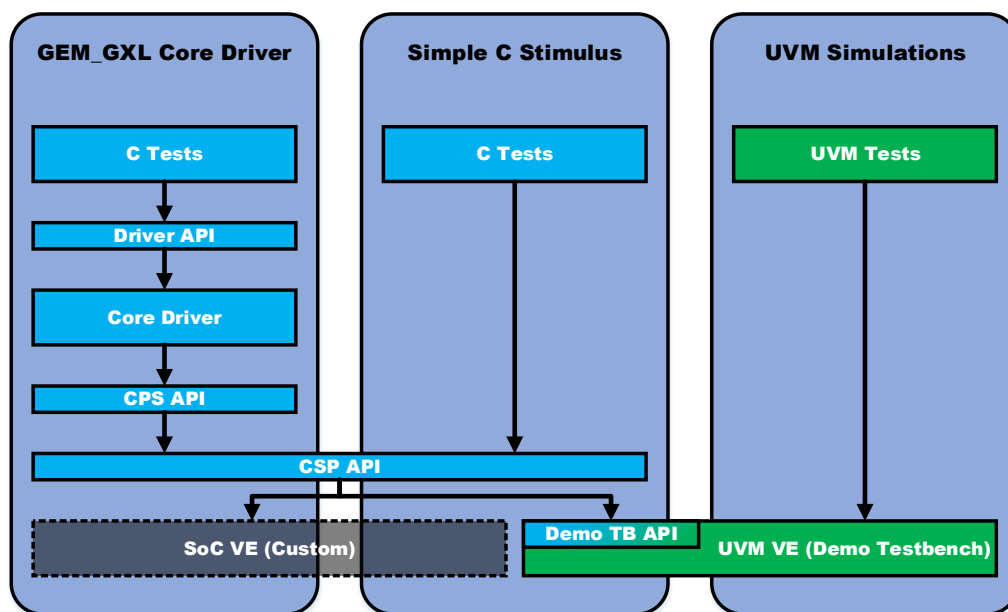


Figure 1: The simulation flows supported in the GEM_GXL Demo Testbench.

The C flows show a software-based testing approach and how it can be reused at system level. This is promoted by conveniently setting the APIs that connects the tests to the relative VE. Using these APIs, C tests are made generic and can run on top of both the GEM_GXL Demo Testbench UVM-SV VE and an SoC VE. This concept has been proved using Cadence CSP¹, to which reference is made throughout this document.

1.1. References

Cadence documentation

- [1] **Cadence.** GEM_GXL Controller IP, User Guide. 2017.
- [2] **Cadence.** Cadence IPs for 2.5G and Gigabit Ethernet MAC, Quick Start Guide. 2017
- [3] **Cadence.** GEM_GXL Ethernet Controller Demo Testbench User Guide. 2017
- [4] **Cadence.** EMAC FW Driver User Guide. 2016.
- [5] **Cadence.** Driver Porting Guide. 2016.

¹ The Configurable System Platform (CSP) is one of the Cadence SoC VE solutions.

2. Architecture

The architecture of the C layer is shown in Figure 2. C tests are described in [3] and will not be further addressed in here. However, they can be found in the following directory:

- <DEMO_TB_INSTALL_PATH>/verif/uvc_lib/cdn_gem_demo/sve/tests_c

The remaining part of the C layer is comprised of both reusable code and VE-dependent code. The reusable code consists in test support source and headers, core driver source and headers and the registers header. Those and the UVM-SV-dependent code are provided in the GEM_GXL Demo Testbench package inside the following directories:

- <DEMO_TB_INSTALL_PATH>/verif/uvc_lib/cdn_demo/c
- <DEMO_TB_INSTALL_PATH>/verif/uvc_lib/cdn_gem_demo/c

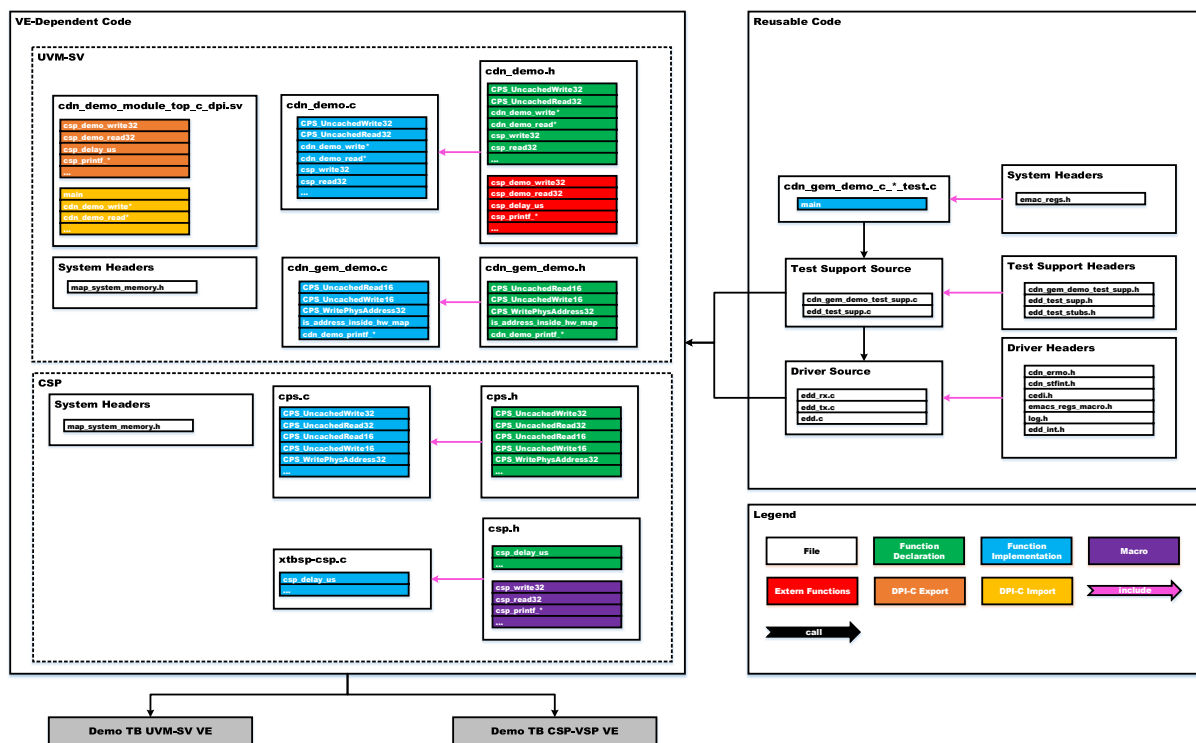


Figure 2: C layer architecture.

The reusable C code uses three APIs: Demo Testbench, CPS² and CSP. The CPS API represents a platform-specific code that connect a given Cadence driver to the rest of the underlying system [4][5]. Not all of the features of CPS are required to operate the GEM_GXL Demo Testbench. The CSP API provides functions for the tests to call in order to communicate with the underlying VEs.

The CPS and CSP APIs must be shared between whichever VE desires to run the GEM_GXL Demo Testbench reusable C code. In order to do this, a VE-dependent implementation must be provided. Please note that the customer will be responsible of VE-dependent code implementation in his own VE, in order to accommodate CPS and CSP APIs. However, the implementation burden is contained and examples are given in the GEM_GXL Demo Testbench package for the UVM-SV VE and in this document for the GEM_GXL Demo Testbench Cadence CSP VE.

² The Cadence Platform Services (CPS) is the API employed by the Cadence core driver [4][5].

The GEM_GXL Demo Testbench Cadence CSP VE uses a Cadence Tensilica Xtensa CPU and an instance of the GEM_GXL IP having the link-side interface connected in loopback.

Base addresses and segment extensions of the various cores are contained inside the `map_system_memory.h` header. This is VE-dependent and shall be implemented by the customer as pointed out above. An example can be found in the GEM_GXL Demo Testbench package.

2.1. C Layer Reuse in UVM-SV

C tests run over the UVM-SV VE using the Demo Testbench API. In addition, C tests exploit an implementation of some functions in the CPS and CSP API. These use several import/export calls across the C/SV language barrier using the C-DPI³ interface.

The most important functions in the Demo Testbench API are:

- **cdn_demo_write***: declared in `cdn_demo.h`, implemented in `cdn_demo.c`. Used from the UVM world to write the C-allocated AXI memory (C-DPI import in `cdn_demo_module_top_c_dpi.sv`).
- **cdn_demo_read***: declared in `cdn_demo.h`, implemented in `cdn_demo.c`. Used from the UVM world to read the C-allocated AXI memory (C-DPI import in `cdn_demo_module_top_c_dpi.sv`).
- **csp_demo_write***: declared in `cdn_demo.h`, implemented in `cdn_demo_module_top_c_dpi.sv`. Used from the C world to write APB registers in UVM (DPI-C export from `cdn_demo_module_top_c_dpi.sv`).
- **csp_demo_read***: declared in `cdn_demo.h`, implemented in `cdn_demo_module_top_c_dpi.sv`. Used from the C world to read APB registers in UVM (C-DPI export from `cdn_demo_module_top_c_dpi.sv`).
- **cdn_demo_printf_***: declared in `cdn_gem_demo.h`, implemented in `cdn_gem_demo.c`. Wrap around `csp_printf_*` and allows them to print a formatted string.

The functions needed from the CPS API are:

- **CPS_UncachedWrite32**: performs a 32-bit uncached write access to the memory. Declared in `cdn_demo.h`, implemented in `cdn_demo.c`. Since the core driver uses this function to access both descriptors and registers, the provided implementation use a function pointer (`fptr_is_addr_inside_hw_map`), which is attached to `is_addr_inside_hw_map`.
- **CPS_UncachedRead32**: performs a 32-bit uncached read access to the memory. Declared in `cdn_demo.h`, implemented in `cdn_demo.c`. Since the core driver uses this function to access both descriptors and registers, the provided implementation use a function pointer (`fptr_is_addr_inside_hw_map`), which is attached to `is_addr_inside_hw_map`.
- **CPS_UncachedWrite16**: performs a 16-bit uncached write access to the memory. Declared in `cdn_gem_demo.h`, implemented in `cdn_gem_demo.c`.
- **CPS_UncachedRead16**: performs a 16-bit uncached read access to the memory. Declared in `cdn_gem_demo.h`, implemented in `cdn_gem_demo.c`.
- **CPS_WritePhysAddress32**: writes a 32-bit address value to memory, bypassing the cache. Declared in `cdn_gem_demo.h`, implemented in `cdn_gem_demo.c`.

The functions needed from the CSP API are:

- **csp_write32**: employed for APB write accesses. Declared in `cdn_demo.h`, implemented in `cdn_demo.c`. Wraps around `csp_demo_write32`, which is a C-DPI export from `cdn_demo_module_top_c_dpi.sv`.
- **csp_read32**: employed for APB read accesses. Declared in `cdn_demo.h`, implemented in `cdn_demo.c`. Wraps around `csp_demo_read32`, which is a C-DPI export from `cdn_demo_module_top_c_dpi.sv`.

³ The SystemVerilog Direct Programming Interface (DPI) is a lightweight interface designed to map SystemVerilog tasks and functions to foreign language implementation (as C) and vice versa.

- **csp_delay_us**: waits for a certain time in microseconds. Declared in `cdn_demo.h`, implemented in `cdn_demo_module_top_c_dpi.sv` (C-DPI export).
- **csp_printf_***: print messages with various severity (error, warning, info and debug). Declared in `cdn_demo.h`, implemented in `cdn_demo_module_top_c_dpi.sv` (C-DPI export). They can only print an already formatted string since the ``uvm_*` macros are used straight away in the implementation.

2.2.C Layer Reuse in SoC

In order for C tests to be reused, the function listed in section 2.1 for the CPS and CSP APIs need a C declaration/implementation in the SoC VE. For the GEM_GXL Demo Testbench Cadence CSP VE, this is reported in the following.

The needed CPS functions are declared in `cps.h` and implemented in `cps.c` as:

```
/* cps.h */

#ifndef _CPS_H_
#define _CPS_H_

#include <stdint.h>

extern uint16_t CPS_UncachedRead16(volatile uint16_t* address);
extern uint32_t CPS_UncachedRead32(volatile uint32_t* address);
extern void CPS_UncachedWrite16(volatile uint16_t* address, uint16_t value);
extern void CPS_UncachedWrite32(volatile uint32_t* address, uint32_t value);
extern void CPS_WritePhysAddress32(volatile uint32_t* location, uint32_t addrValue);

#endif
```

```
/* cps.c */

#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include "cps.h"

uint16_t CPS_UncachedRead16(volatile uint16_t* address) {
    return *address;
}

uint32_t CPS_UncachedRead32(volatile uint32_t* address) {
    return *address;
}

void CPS_UncachedWrite16(volatile uint16_t* address, uint16_t value) {
    *address = value;
}

void CPS_UncachedWrite32(volatile uint32_t* address, uint32_t value) {
    *address = value;
}

void CPS_WritePhysAddress32(volatile uint32_t* location, uint32_t addrValue) {
    *location = addrValue;
}
```

The needed CSP functions are contained in `csp.h`. The function `csp_delay_us` is CPU-specific and hence its implementation cannot be shown in detail here. It is left to the customer to find a suitable implementation for his own SoC-VE as per CSP API specifications.

```
/* csp.h */

#ifndef _CSP_H_
#define _CSP_H_

#include "cps.h"

#define csp_read32(addr) \
    (*((volatile uint32_t *) (addr)))

#define csp_write32(addr, data) \
    (*((volatile uint32_t *) (addr)) = (data))

#define csp_printf(format, ...) \
    printf(format, ##__VA_ARGS__)

#define csp_printf_error(format, ...) \
    csp_printf("ERROR: " format, ##__VA_ARGS__)

#define csp_printf_warning(format, ...) \
    csp_printf("WARNING: " format, ##__VA_ARGS__)

#define csp_printf_info(format, ...) \
    csp_printf("INFO: " format, ##__VA_ARGS__)

void csp_delay_us(uint32_t us);

#endif
```